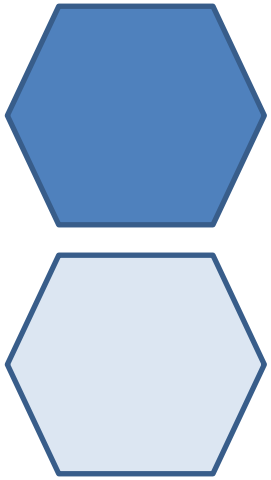
Bioinformatics

Ing. Giulia Fiscon, PhD

fiscon@diag.uniroma1.it

Department of Computer, Control, and Management Engineering
Sapienza University of Rome

- R programming
  Part III: control structures, control functions

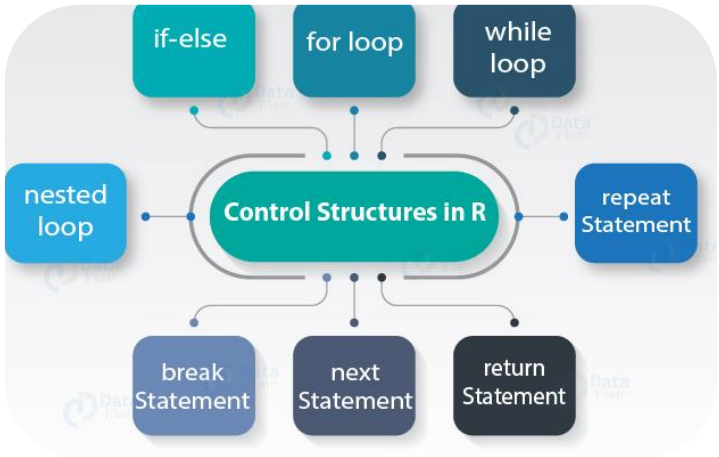


Control structures

Control structures

Control structures in R allow you to control the flow of execution of the program, depending on runtime conditions. Common structures are

- if, else: testing a condition
- for: execute a loop a fixed number of times
- while: execute a loop while a condition is true
- repeat: execute an infinite loop
- break: break the execution of a loop
- next: skip an iteration of a loop
- return: exit a function

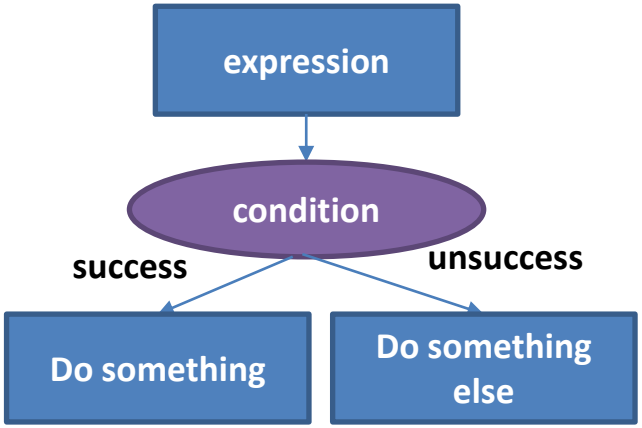


Most control structures are not used in interactive sessions, but rather when writing functions or longer expressions

Control structures: if

Syntax:

```
if (test_expression) {
statement
} else {
statement
}
```



Example:

```
a <- 5
b <- 6
if(a<b){
 print("a is smaller than b")
}
```

```
if(a>b){
  print("a is greater than b")
} else{
  print("b is greater than a")
}
```

```
val1 = 10                                   #Creating our first variable val1
val2 = 5                                    #Creating second variable val2
if (val1 > val2){                           #Executing Conditional Statement based on the comparison
  print("Value 1 is greater than Value 2")
} else if (val1 < val2){
  print("Value 1 is less than Value 2")
}
```

Control structures: for

- for loops take an iterator variable and assign it successive values from a sequence or vector.
- For loops are most commonly used for iterating over the elements of an object (list, vector, etc.)

```
for(i in 1:10) {
print(i)
}
```

This loop takes the `i` variable and in each iteration of the loop gives it values 1, 2, 3, ..., 10, and then exits.

Control structures: for

- These three loops have the same behavior

```
x <- c("a", "b", "c", "d")

for(i in 1:4) {
print(x[i])
}

for(i in seq_along(x)) {
print(x[i])
}

for(letter in x) {
print(letter)
}

for(i in 1:4) print(x[i])
```

- For loop can be nested

```
x <- matrix(1:6, 2, 3)

for(i in 1:nrow(x)) {
          for(j in 1:ncol(x)) {
             print(x[i,j])
          }
}
```

Control structures: while

- While loops begin by testing a condition.
- If it is true, then they execute the loop body.
- Once the loop body is executed, the condition is tested again, and so forth.

```
count <- 0
while(count < 10) {
          print(count)
          count <- count + 1
}
```

While loops can potentially result in infinite loops if not written properly. Use with care!

```
i <- 0
while(i < 10) {
          print(paste("this is iteration number",i))
          i <- i + 1
}
```

Control structures: next, return

- `next` is used to skip an iteration of a loop

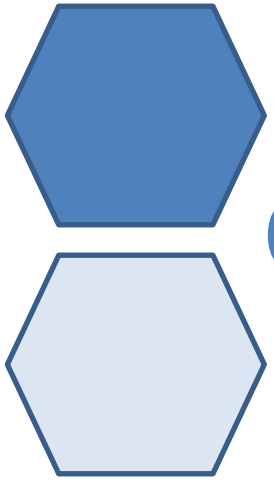
```
for(i in 1:100) {
        if(i <= 30) {
                ## Skip the first 30 iterations
                next
        }
        ## Do something here
}
```

- `return` signals that a function should exit and return a given value. If the return is not specified the function will give the last operation.

```
Add_and_multiple2 <- function(x,y){
     s <- x + y
     p <- x*y
     res <- list(sum = s, product = p)
     return(res)
}
```

Control structures: to sum up

- Control structures like **if**, **while**, and **for** allow you to control the flow of an R program
- Infinite loops should generally be avoided, even if they are theoretically correct.
- Control structures mentioned here are primarily useful for writing programs;
- for command-line interactive work, the ***apply functions** are more useful.



Control functions

Apply family

- How to efficiently apply a function to each element of data frame and list.

  For instance: to apply a function to the rows/columns of a matrix

- Writing for, while loops is useful when programming but not particularly easy when working interactively on the command line. There are some functions which implement looping to make life easier:

  apply: Apply a function over the margins of a matrix

  lapply: Loop over a list and evaluate a function on each element

  sapply: Same as lapply but try to simplify the result

  tapply: Apply a function over subsets of a vector

apply

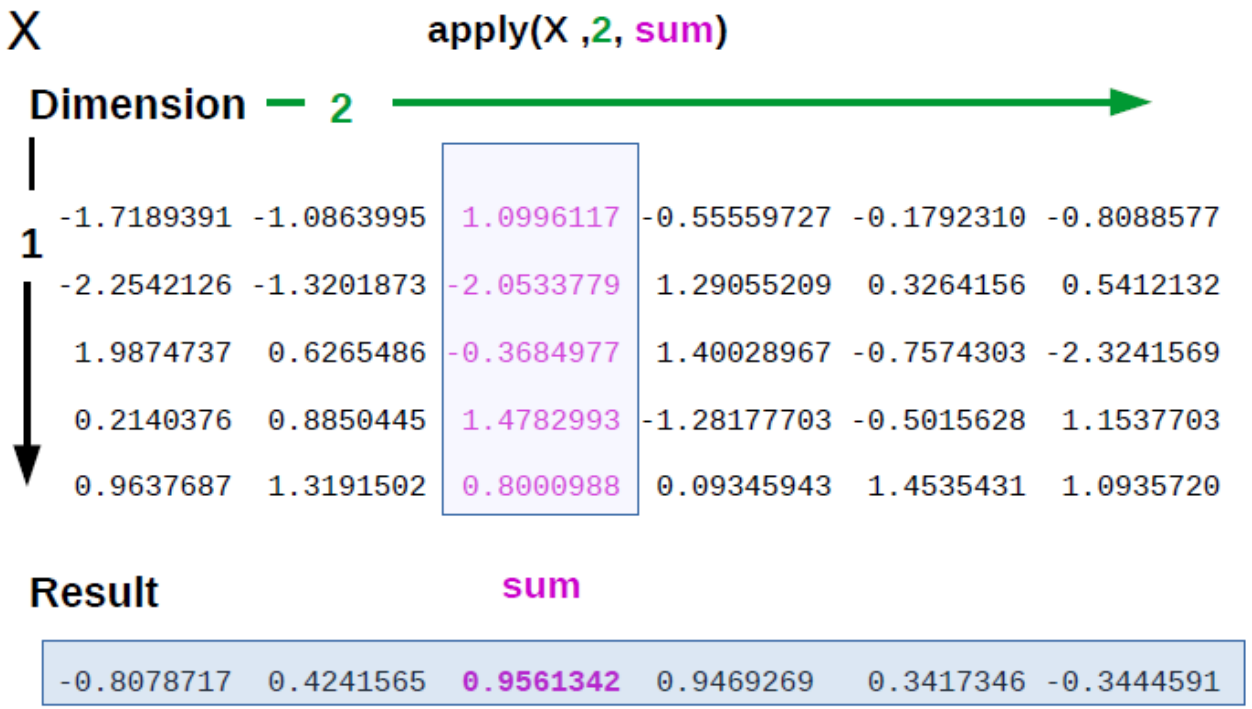
- apply function returns a vector or array of values obtained by applying a function to margins of an array or matrix.
- It is most often used to apply a function to the rows or columns of a matrix
- It is not really faster than writing a loop, but it works in one line!

apply(X, MARGIN, FUN, ...)

X : array;
MARGIN : 1 for rows, 2 for columns;
FUN : one function to be applied;
... : optional arguments to FUN

apply

- apply function returns a vector or array of values obtained by applying a function to margins of an array or matrix.
- It is most often used to apply a function to the rows or columns of a matrix
- It is not really faster than writing a loop, but it works in one line!



apply

- apply function returns a vector or array of values obtained by applying a function to margins of an array or matrix.
- It is most often used to apply a function to the rows or columns of a matrix
- It is not really faster than writing a loop, but it works in one line!

apply(X, MARGIN, FUN, ...)

X : array;
MARGIN : 1 for rows, 2 for columns;
FUN : one function to be applied;
... : optional arguments toFUN

```
> m
            [, 1]       [, 2]
[1, ] −0.1767643 −0.1950407
[2, ] 1.5306045 0.3307676
[3, ] −0.3806768 0.8992097

> apply(m, 1, sum) by rows
[1] − 0.3718050 1.8613721 0.5185329

> apply(m, 2, sum) by columns
[1]0.9731634 1.0349366
```

apply

- apply function returns a vector or array of values obtained by applying a function to margins of an array or matrix.
- It is most often used to apply a function to the rows or columns of a matrix
- It is not really faster than writing a loop, but it works in one line!

apply(X, MARGIN, FUN, ...)

X : array;
MARGIN : 1 for rows, 2 for columns;
FUN : one function to be applied;
... : optional arguments toFUN

```
> m
           [, 1]       [, 2]
[1, ] −0.1767643 −0.1950407
[2, ] 1.5306045 0.3307676
[3, ] −0.3806768 0.8992097

> apply(m, 1, max) by rows
[1] − 0.1767643 1.5306045 0.8992097

> apply(m, 2, max) by columns
[1] − 0.3806768 − 0.1950407
```

apply

Quantile: set of values of a variate which divide a frequency distribution into equal groups, each containing the same fraction of the total population.

Quantiles of the rows of a matrix.

```
> x <- matrix(rnorm(200), 20, 10)
> apply(x, 1, quantile, probs = c(0.25, 0.75))
          [,1]        [,2]       [,3]        [,4]
25% -0.3304284 -0.99812467 -0.9186279 -0.49711686
75%  0.9258157  0.07065724  0.3050407 -0.06585436
           [,5]       [,6]      [,7]       [,8]
25% -0.05999553 -0.6588380 -0.653250 0.01749997
75%  0.52928743  0.3727449  1.255089 0.72318419
          [,9]      [,10]      [,11]      [,12]
25% -1.2467955 -0.8378429 -1.0488430 -0.7054902
75%  0.3352377  0.7297176  0.3113434  0.4581150
         [,13]      [,14]      [,15]      [,16]
25% -0.1895108 -0.5729407 -0.5968578 -0.9517069
75%  0.5326299  0.5064267  0.4933852  0.8868922
         [,17]      [,18]      [,19]     [,20]
```

Inter quartile range (IQR) of the rows of a matrix

```
> apply(x,1,IQR)

 [1] 1.5771539 1.2017801 1.2872311 1.2442468 0.8530182 1.2852498
1.5432260 1.2640521 1.7925931 0.9512165 2.0495357 2.9642476 1.2936809
1.5157700
[15] 0.9175093 1.3912280 0.7854083 1.2429175 1.2083855 0.7055743
```

col/row sums and means

For sums and means of matrix dimensions, we have some shortcuts.

- `rowSums` = apply(x, 1, sum)
- `rowMeans` = apply(x, 1, mean)
- `colSums` = apply(x, 2, sum)
- `colMeans` = apply(x, 2, mean)

The shortcut functions are *much* faster, but you won't notice unless you're using a large matrix.

lapply

- the lapply function returns a list where each element is the result of applying a function to the corresponding element of input data structure.

lapply(X, FUN, ...)
X : any data that can be compatible with a list;
FUN : one function to be applied;
... : optional arguments to FUN;

- lapply() function does not need MARGIN.

lapply

- A very easy example can be to change the string value of a matrix to lower case with tolower function.

```
movies <- c("SPYDERMAN","BATMAN","VERTIGO","CHINATOWN")
movies_lower <-lapply(movies, tolower)
str(movies_lower)
```

Output:

```
## List of 4
## $:chr"spyderman"
## $:chr"batman"
## $:chr"vertigo"
## $:chr"chinatown"
```

We can use unlist() to convert the list into a vector.

```
movies_lower <-unlist(lapply(movies,tolower))
str(movies_lower)
```

Output:

```
##  chr [1:4] "spyderman" "batman" "vertigo" "chinatown"
```

lapply

- the lapply function returns a list where each element is the result of applying a function to the corresponding element of input data structure.

lapply(X, FUN, ...)
X : any data that can be compatible with a list;
FUN : one function to be applied;
... : optional arguments to FUN;

> *data*() **to list in-built data set**
> *lapply*(*trees, mean*) **trees is a in-built data set**

$Girth
[1] 13.24839

$Height
[1] 76

$Volume
[1] 30.17097

```
> trees
   Girth Height Volume
1    8.3     70   10.3
2    8.6     65   10.3
3    8.8     63   10.2
4   10.5     72   16.4
5   10.7     81   18.8
6   10.8     83   19.7
7   11.0     66   15.6
8   11.0     75   18.2
9   11.1     80   22.6
10  11.2     75   19.9
11  11.3     79   24.2
12  11.4     76   21.0
13  11.4     76   21.4
14  11.7     69   21.3
15  12.0     75   19.1
16  12.9     74   22.2
17  12.9     85   33.8
18  13.3     86   27.4
19  13.7     71   25.7
20  13.8     64   24.9
21  14.0     78   34.5
22  14.2     80   31.7
23  14.5     74   36.3
24  16.0     72   38.3
25  16.3     77   42.6
26  17.3     81   55.4
27  17.5     82   55.7
28  17.9     80   58.3
29  18.0     80   51.5
30  18.0     80   51.0
31  20.6     87   77.0
```

sapply

- the sapply function is a user-friendly version and wrapper of "lapply" by default returning a vector, matrix

sapply(X, FUN, ...)
X : any data that can be compatible with a list/vector/matrix;
FUN : one function to be applied;
... : optional arguments to FUN;

> *data*() **to list in-built data set**
> *sapply*(*trees, mean*) **trees is a in-built data set**

Girth	***Height***	***Volume***
13.24839	76.00000	30.17097

```
> trees
   Girth Height Volume
1    8.3     70   10.3
2    8.6     65   10.3
3    8.8     63   10.2
4   10.5     72   16.4
5   10.7     81   18.8
6   10.8     83   19.7
7   11.0     66   15.6
8   11.0     75   18.2
9   11.1     80   22.6
10  11.2     75   19.9
11  11.3     79   24.2
12  11.4     76   21.0
13  11.4     76   21.4
14  11.7     69   21.3
15  12.0     75   19.1
16  12.9     74   22.2
17  12.9     85   33.8
18  13.3     86   27.4
19  13.7     71   25.7
20  13.8     64   24.9
21  14.0     78   34.5
22  14.2     80   31.7
23  14.5     74   36.3
24  16.0     72   38.3
25  16.3     77   42.6
26  17.3     81   55.4
27  17.5     82   55.7
28  17.9     80   58.3
29  18.0     80   51.5
30  18.0     80   51.0
31  20.6     87   77.0
```

sapply

- sapply will try to simplify the result of lapply if possible.
- If the result is a list where every element is length 1, then a vector is returned
- If the result is a list where every element is a vector of the same length (> 1), a matrix is returned.
- If it can't figure things out, a list is returned

split

- split takes a vector or other objects and splits it into groups determined by a factor or list of factors

split (x, f, drop = FALSE, ...)
x: is a vector (or list) or data frame to be split
f: is a factor (or coerced to one) or a list of factors according to which x should be divided
Drop: logical value that indicates whether empty factors levels should be dropped

split

```
> x <- c(rnorm(10), runif(10), rnorm(10, 1))
> f <- gl(3, 10)
> split(x, f)
$`1`
[1] -0.8493038 -0.5699717 -0.8385255 -0.8842019
[5] 0.2849881 0.9383361 -1.0973089 2.6949703
[9] 1.5976789 -0.1321970
$`2`
[1] 0.09479023 0.79107293 0.45857419 0.74849293
[5] 0.34936491 0.35842084 0.78541705 0.57732081
[9] 0.46817559 0.53183823
$`3`
[1] 0.6795651 0.9293171 1.0318103 0.4717443
[5] 2.5887025 1.5975774 1.3246333 1.4372701
```

split

- A common idiom is split followed by a lapply

```
> lapply(split(x, f), mean)
$`1`
[1] 0.1144464
$`2`
[1] 0.5163468
$`3`
[1] 1.246368
```

Split a data.frame

```
> library(datasets)
> head(airquality)
Ozone Solar.R Wind Temp Month Day
1 41 190 7.4 67 5 1
2 36 118 8.0 72 5 2
3 12 149 12.6 74 5 3
4 18 313 11.5 62 5 4
5 NA NA 14.3 56 5 5
6 28 NA 14.9 66 5 6
```

Split a data.frame

```
> s <- split(airquality, airquality$Month)

> lapply(s, function(x) colMeans(x[, c("Ozone", "Solar.R",
"Wind")]))
$`5`
Ozone Solar.R Wind
NA NA 11.62258
$`6`
Ozone Solar.R Wind
NA 190.16667 10.26667
$`7`
Ozone Solar.R Wind
NA 216.483871 8.941935
```

Split a data.frame

- Simplify the result with sapply

```
> sapply(s, function(x) colMeans(x[, c("Ozone", "Solar.R",
"Wind")]))
5 6 7 8 9
Ozone NA NA NA NA NA
Solar.R NA 190.16667 216.483871 NA 167.4333
Wind 11.62258 10.26667 8.941935 8.793548 10.1800

> sapply(s, function(x) colMeans(x[, c("Ozone", "Solar.R",
"Wind")],
na.rm = TRUE))
5 6 7 8 9
Ozone 23.61538 29.44444 59.115385 59.961538 31.44828
Solar.R 181.29630 190.16667 216.483871 171.857143 167.43333
Wind 11.62258 10.26667 8.941935 8.793548 10.18000
```

Split on more than one level

- Split using two classification categories (f1,f2)

```
> x <- rnorm(10)
> f1 <- gl(2, 5)
> f2 <- gl(5, 2)
> f1
[1] 1 1 1 1 1 2 2 2 2 2
Levels: 1 2
> f2
[1] 1 1 2 2 3 3 4 4 5 5
Levels: 1 2 3 4 5
> interaction(f1, f2)
[1] 1.1 1.1 1.2 1.2 1.3 2.3 2.4 2.4 2.5 2.5
10 Levels: 1.1 2.1 1.2 2.2 1.3 2.3 1.4 ... 2.5
```

Split on more than one level

- Split using two classification categories (f1,f2)

```
> str(split(x, list(f1, f2)))
List of 10
$ 1.1: num [1:2] -0.378 0.445
$ 2.1: num(0)
$ 1.2: num [1:2] 1.4066 0.0166
$ 2.2: num(0)
$ 1.3: num -0.355
$ 2.3: num 0.315
$ 1.4: num(0)
$ 2.4: num [1:2] -0.907 0.723
$ 1.5: num(0)
$ 2.5: num [1:2] 0.732 0.360
```

Split on more than one level

- Using drop = TRUE, empty levels will be cut down

```
> str(split(x, list(f1, f2), drop = TRUE))
List of 6
$ 1.1: num [1:2] -0.378 0.445
$ 1.2: num [1:2] 1.4066 0.0166
$ 1.3: num -0.355
$ 2.3: num 0.315
$ 2.4: num [1:2] -0.907 0.723
$ 2.5: num [1:2] 0.732 0.360
```

tapply

- the tapply function allows us to apply a function on a subset of values grouped according to one or more factors

tapply(X, INDEX, FUN, ...)
X : any data that can be compatible with a list;
INDEX : list of one or more factors used to cluster X;
FUN : one function to be applied;
... : optional arguments to FUN;

> *library*(*MASS*) **to load MASS data set**
> *Cars*93 **Car93 is a MASS data set**

> *tapply*(*Cars*93$*Price, Cars*93$*Manufacturer, mean*) **Compute the average price for each brand**

- the tapply function combines the split function and the apply function to apply a particular function on subsets of data

tapply

- the tapply function allows us to apply a function on a subset of values grouped according to one or more factors

tapply(X, INDEX, FUN, ...)
X : any data that can be compatible with a list;
INDEX : list of one or more factors used to cluster X;
FUN : one function to be applied;
... : optional arguments to FUN;

- For example, the iris dataset collects information for each species about their length and width. We can compute the median of the length for each species

```
data(iris)
tapply(iris$Sepal.Width, iris$Species, median)
```

Output:

```
##     setosa versicolor  virginica
##        3.4        2.8        3.0
```

Control functions: to sum up

Function	Arguments	Objective	Input	Output
apply	apply(x, MARGIN, FUN)	Apply a function to the rows or columns or both	Data frame or matrix	vector, list, array
lapply	lapply(X, FUN)	Apply a function to all the elements of the input	List, vector or data frame	list
sapply	sapply(X, FUN)	Apply a function to all the elements of the input	List, vector or data frame	vector or matrix

Exercises on apply

- Compute sums of the columns of the hills data set;
- Compute row and column sums of a matrix 10x10 whose values are generated according to uniform distribution between 4 and 10;
- Use apply to calculate the standard deviation of the columns of a matrix;
- Consider in-built data set "airquality" compute the average wind speed and ozone percentage with respect to "month" column.

Exercises on apply

- Compute sums of the columns of the hills data set

```
> library(MASS)

> lapply(hills, sum)

$dist
[1] 263.5

$climb
[1] 63536

$time
[1] 2025.65
> sapply(hills, sum)

   dist    climb     time
 263.50 63536.00  2025.65
```

Exercises on apply

- Compute row and column sums of a matrix 10x10 whose values are generated according to uniform distribution between 4 and 10;

> m = matrix(*runif* (100*, min* = 4*, max* = 10)*, ncol* = 10)

> c1 = colSums(m)
The same of *c2 = apply(m,2,sum)*

> r1 = rowSums(m)
The same of *r2 = apply(m,1,sum)*

Exercises on apply

Use apply to calculate the standard deviation of the columns of a matrix.

```
> m = matrix(runif (100, min = 4, max = 10), ncol = 10)

> apply(m, 2, sd)
```

Exercises on apply

Consider in-built data set "airquality" compute the average wind speed and ozone percentage with respect to "month" column.

> tapply(airquality$Wind, airquality$Month, mean)

> tapply(airquality$Ozone, airquality$Month, mean, na.rm = TRUE)

na.rm=TRUE removes NA from mean computation

Exercise on apply

- Create a spreadsheet with 4 columns:
  - Age → free-choice of integers
  - Gender → Male/Female (1° classification criterion)
  - Hair colour→ blond/dark (2°classification criterion)
  - Wear glasses → yes/no (3°classification criterion)

- Save the spreadsheet as txt file and import it in R by using **read.table** function
- Calculate the average for each combination of the three classification criteria

- Hint: use the **split** function to divide the matrix into sub-groups on the basis of the three classification levels and use the appropriate function of the apply group to obtain the average age of each sub-group

Solution

```
df <- read.table("df.txt", header = T, sep="\t", quote = "", check.names = F)

##### split + sapply
s <- split(df$age,df$hair, drop = T)
result1 <- sapply(s,mean)

s <- split(df$age,df$glasses, drop = T)
result2 <- sapply(s,mean)

s <- split(df$age,df$gender, drop = T)
result3 <- sapply(s,mean)

c(result1,result2,result3)

##### or tapply
result1 <- tapply(df$age, df$hair, mean)
result2 <- tapply(df$age, df$glasses, mean)
result3 <- tapply(df$age, df$gender, mean)
c(result1,result2,result3)

#or
result <- lapply(names(df)[2:4],function(x){
  s <- split(df[,1],df[x], drop = T)
  result <- sapply(s,mean)
})
names(result) <- names(df)[2:4]
```

Solution

```
df <- read.table("df.txt", header = T, sep="\t", quote = "", check.names = F)

##### all possibile combination of classification criteria
###### split + lapply
s <- split(df$age, list(df$gender, df$hair, df$glasses), drop = T)
res <- lapply(s,mean)

$M.blond.no
[1] 71.5

$F.dark.no
[1] 78.25

$M.dark.no
[1] 82

$F.blond.yes
[1] 79.5
```